M14 Architectural Opinion: Platform Engine & Template Separation

> **Verdict: I AGREE with the audit.** Proceed with implementation. **Date:** 2025-07-24 **Requested by:** Team (external audit follow-up) **Scope:** Platform/Template separation + Decisions separation

1. Audit Claims vs. Code Evidence

Claim 1: "The state machine is domain-agnostic"

VERIFIED — TRUE. (`src/webapp/orchestrator/state-machine.js`)

- `buildTransitionMap(phases)` accepts ANY phases array — no SDLC vocabulary
- Phase names are generic (`PHASE_1`–`PHASE_4`, not "Requirements & Strategy")
- `isPhaseOrMatchingCritic()` pattern-matches CRITIC_N to PHASE_N — purely structural, works for any N-phase pipeline
- **Exception:** `MODE_CONFIGS` (9 modes) IS hardcoded but could trivially be loaded from YAML (the data structure is already a simple object)

Claim 2: "The dispatcher is a mechanical lookup"

VERIFIED — TRUE. (`src/webapp/orchestrator/dispatcher.js`)

- `PHASE_AGENTS` is a plain frozen object mapping states to `{id, name}` arrays
- Zero domain semantics in the dispatch logic — it reads the map, loads the skill file, injects context, and invokes
- `skillsDir` and `docsDir` are already configurable via `DEFAULT_CONFIG`
- The ONLY SDLC-specific content is the PHASE_AGENTS data itself

Claim 3: "The contract/guardrail system is reusable"

VERIFIED — TRUE. (`src/webapp/orchestrator/gate-validator.js`)

- Validation logic (checking `UNCERTAIN:`, `INSUFFICIENT_DATA:`, handoff checklists, placeholder detection) is entirely domain-agnostic
- `CONTRACTS_DIR` and `GUARDRAILS_DIR` are already overridable via constructor options
- `PHASE_CONTRACTS` and `PHASE_GUARDRAILS` mappings are hardcoded but the validation engine doesn't care what the contracts contain
- All 25 contracts follow the same meta-pattern: PURPOSE → OUTPUT FILES → MANDATORY SECTIONS

Additional Findings

Component	Domain-Independent?	Evidence
`engine.js`	**100% generic**	Pure integration: wires state machine + flow loader + persistence + SSE

Component	Domain-Independent?	Evidence
`flow-loader.js`	**100% generic**	Parses ANY flows.yaml structure
`state-persistence.js`	**100% generic**	Abstract store interface, merge-on-write
`sprint-gate.js`	**~80% generic**	Gate checking is generic; sprint ID format is SDLC-specific
`flows.yaml`	**SDLC-specific**	Defines SDLC phases, modes, and transitions
`platform/schema/`	**Already extracted**	Canonical schemas exist with agents.json, flows.json, tools.json

Overall assessment: ~60% of the orchestrator is domain-agnostic engine code. ~40% is SDLC-specific data that drives the engine.

---

2. Evaluation of Team Requirements

Requirement: "Single repo, not commercial"

Sound decision. For an open-source project, a monorepo avoids npm/package publishing overhead, version matrix management, and dependency hell. The platform and templates can be colocated with clear directory boundaries.

Requirement: "`platform/` folder for framework"

Sound decision. `platform/schema/` already exists (created during M4 canonical schema work). This is evidence the architecture was already heading in this direction. Extending to `platform/engine/` is natural.

Requirement: "`templates/` folder for SDLC (and future templates)"

Sound decision. A template pack bundles: agents, contracts, guardrails, playbooks, flows.yaml, and a manifest.json. Each pack is self-contained. The SDLC pack would be at `templates/sdlc/`.

Requirement: "Template selection before onboarding"

Feasible. The engine already reads mode from `session-state.json`. Adding `template: 'sdlc'` to session state is trivial. The webapp can present a template picker before issuing the first CREATE/AUDIT command. The engine loads the selected template's manifest and configures itself.

---

3. Decisions Separation Assessment

I AGREE this should be done.

Current state:

- `BusinessDocs/decisions.md` — 245 decisions across 20 category files, all focused on technology stack choices for the solution being built

- `docs/help/decisions-architecture.md` — system architecture documentation

The problem: when a user creates a new project with a *different* template, they would inherit SDLC-specific decision categories that don't apply. Decision categories should be template-provided, not platform-baked.

Proposed separation:

- **Platform decisions** → `platform/decisions/` (how the engine is configured, architectural choices about the framework itself)
- **Template decisions** → template packs provide default decision categories (e.g., SDLC template provides technology stack decisions)
- **Project decisions** → `BusinessDocs/decisions/` (user answers, project-specific)

---

4. Proposed Target Architecture

```
platform/
  engine/              ← state-machine.js, dispatcher.js, gate-validator.js,
  │                      engine.js, flow-loader.js, state-persistence.js,
  │                      sprint-gate.js, cli.js, + supporting files
  schema/              ← already exists (canonical JSON schemas)
  decisions/           ← platform-level architectural decisions
  README.md

templates/
  sdlc/
    agents/            ← 38 agent skill files (00-orchestrator.md through 37-*)
    contracts/         ← 25 output contracts
    guardrails/        ← 11 guardrail rule sets
    playbooks/         ← playbook files
    flows.yaml         ← SDLC-specific flow definition
    manifest.json      ← template metadata + agent registry + contract/guardrail
mappings
    copilot-rules.md   ← template-level system instructions
    README.md

src/webapp/            ← stays (presentation layer: server, routes, UI)
BusinessDocs/          ← stays (per-project output)
docs/                  ← platform documentation (help, reference, guides)
tests/                 ← stays (tests reference platform/engine/ after update)
```

---

5. Risk Assessment

Risk	Impact	Likelihood	Mitigation
Path breakage during migration	HIGH	HIGH	Strict dependency ordering of stories; full test run after each story

Risk	Impact	Likelihood	Mitigation
copilot-instructions.md desync	HIGH	MEDIUM	Dedicated story for documentation; semantic validation
Agent skill files reference old paths	MEDIUM	HIGH	Bulk search-replace; agent files reference contracts/guardrails by path
CI pipeline breaks	MEDIUM	MEDIUM	Dedicated Docker/CI story; test in feature branch before merge
Template loader introduces new bugs	MEDIUM	LOW	Well-tested template loader (S1); existing engine tests validate behavior
Scope creep	MEDIUM	MEDIUM	Strict story boundaries; no feature additions beyond separation

6. What I Would Change vs. Audit Recommendations

The audit is sound, but I'd make two adjustments:

1. **Don't move `src/webapp/` into `platform/`.** The webapp is a presentation layer that USES the engine — it's not the engine itself. Keeping `src/webapp/` separate maintains the clean layering: `platform/engine/` (logic) → `src/webapp/` (presentation) → `templates/` (content).
2. **Don't create a meta-contract template yet.** The audit suggests condensing 25 contracts to 1 template + parameters. This is a good idea long-term but adds scope to M14. Each contract has nuanced requirements. Do this as a follow-up milestone.

HANDOFF CHECKLIST

- ☑ All required sections are filled (not empty, not placeholder)
- ☑ All findings include source references (file paths, line numbers)
- ☑ No UNCERTAIN: items — all claims verified against code
- ☑ No contradictory statements
- ☑ Risk assessment includes mitigations
- ☑ Deliverable written to file per MEMORY MANAGEMENT PROTOCOL